

Chapter 1. Introduction

Welcome to Programming

I love programming. I enjoy the challenge to not only make a working program, but to do so with style. Programming is like poetry. It conveys a message, not only to the computer, but to those who modify and use your program. With a program, you build your own world with your own rules. You create your world according to your conception of both the problem and the solution. Masterful programmers create worlds with programs that are clear and succinct, much like a poem or essay.

One of the greatest programmers, Donald Knuth, describes programming not as telling a computer how to do something, but telling a person how they would instruct a computer to do something. The point is that programs are meant to be read by people, not just computers. Your programs will be modified and updated by others long after you move on to other projects. Thus, programming is not as much about communicating to a computer as it is communicating to those who come after you. A programmer is a problem-solver, a poet, and an instructor all at once. Your goal is to solve the problem at hand, doing so with balance and taste, and teach your solution to future programmers. I hope that this book can teach at least some of the poetry and magic that makes computing exciting.

Most introductory books on programming frustrate me to no end. At the end of them you can still ask "how does the computer really work?" and not have a good answer. They tend to pass over topics that are difficult even though they are important. I will take you through the difficult issues because that is the only way to move on to masterful programming. My goal is to take you from knowing nothing about programming to understanding how to think, write, and learn like a programmer. You won't know everything, but you will have a background for how everything fits together. At the end of this book, you should be able to do the following:

Chapter 1. Introduction

- Understand how a program works and interacts with other programs
- Read other people's programs and learn how they work
- Learn new programming languages quickly
- Learn advanced concepts in computer science quickly

I will not teach you everything. Computer science is a massive field, especially when you combine the theory with the practice of computer programming. However, I will attempt to get you started on the foundations so you can easily go wherever you want afterwards.

There is somewhat of a chicken and egg problem in teaching programming, especially assembly language. There is a lot to learn - it's almost too much to learn almost at once, but each piece depends on all the others. Therefore, you must be patient with yourself and the computer while learning to program. If you don't understand something the first time, reread it. If you still don't understand it, it is sometimes best to take it by faith and come back to it later. Often after more exposure to programming the ideas will make more sense. Don't get discouraged. It's a long climb, but very worthwhile.

At the end of each chapter are three sets of review exercises. The first set is more or less regurgitation - they check to see if you can give back what you learned in the chapter. The second set contains application questions - they check to see if you can apply what you learned to solve problems. The final set is to see if you are capable of broadening your horizons. Some of these questions may not be answerable until later in the book, but they give you some things to think about. Other questions require some research into outside sources to discover the answer. Still others require you to simply analyze your options and explain a best solution. Many of the questions don't have right or wrong answers, but that doesn't mean they are unimportant. Learning the issues involved in programming, learning how to research answers, and learning how to look ahead are all a major part of a programmer's work.

If you have problems that you just can't get past, there is a mailing list for this

book where readers can discuss and get help with what they are reading. The address is `pgubook-readers@nongnu.org`. This mailing list is open for any type of question or discussion along the lines of this book. You can subscribe to this list by going to <http://mail.nongnu.org/mailman/listinfo/pgubook-readers>.

Your Tools

This book teaches assembly language for x86 processors and the GNU/Linux operating system. Therefore we will be giving all of the examples using the GNU/Linux standard GCC tool set. If you are not familiar with GNU/Linux and the GCC tool set, they will be described shortly. If you are new to Linux, you should check out the guide available at <http://rute.sourceforge.net/>¹ What I intend to show you is more about programming in general than using a specific tool set on a specific platform, but standardizing on one makes the task much easier.

Those new to Linux should also try to get involved in their local GNU/Linux User's Group. User's Group members are usually very helpful for new people, and will help you from everything from installing Linux to learning to use it most efficiently. A listing of GNU/Linux User's Groups is available at <http://www.linux.org/groups/>

All of these programs have been tested using Red Hat Linux 8.0, and should work with any other GNU/Linux distribution, too.² They will not work with non-Linux operating systems such as BSD or other systems. However, all of the *skills* learned in this book should be easily transferable to any other system.

If you do not have access to a GNU/Linux machine, you can look for a hosting provider who offers a Linux *shell account*, which is a command-line only interface

1. This is quite a large document. You certainly don't need to know everything to get started with this book. You simply need to know how to navigate from the command line and how to use an editor like `pico`, `emacs`, or `vi` (or others).

2. By "GNU/Linux distribution", I mean an x86 GNU/Linux distribution. GNU/Linux distributions for the Power Macintosh, the Alpha processor, or other processors will not work with this book.

Chapter 1. Introduction

to a Linux machine. There are many low-cost shell account providers, but you have to make sure that they match the requirements above (i.e. - Linux on x86). Someone at your local GNU/Linux User's Group may be able to give you one as well. Shell accounts only require that you already have an Internet connection and a telnet program. If you use Windows®, you already have a telnet client - just click on `start`, then `run`, then type in `telnet`. However, it is usually better to download PuTTY from <http://www.chiart.greenend.co.uk/~sgtatham/putty/> because Windows' telnet has some weird problems. There are a lot of options for the Macintosh, too. NiftyTelnet is my favorite.

If you don't have GNU/Linux and can't find a shell account service, then you can download Knoppix from <http://www.knoppix.org/> Knoppix is a GNU/Linux distribution that boots from CD so that you don't have to actually install it. Once you are done using it, you just reboot and remove the CD and you are back to your regular operating system.

So what is GNU/Linux? GNU/Linux is an operating system modeled after UNIX®. The GNU part comes from the GNU Project (<http://www.gnu.org/>)³, which includes most of the programs you will run, including the GCC tool set that we will use to program with. The GCC tool set contains all of the programs necessary to create programs in various computer languages.

Linux is the name of the *kernel*. The kernel is the core part of an operating system that keeps track of everything. The kernel is both an fence and a gate. As a gate, it allows programs to access hardware in a uniform way. Without the kernel, you would have to write programs to deal with every device model ever made. The kernel handles all device-specific interactions so you don't have to. It also handles file access and interaction between processes. For example, when you type, your typing goes through several programs before it hits your editor. First, the kernel is what handles your hardware, so it is the first to receive notice about the keypress. The keyboard sends in *scan codes* to the kernel, which then converts them to the actual letters, numbers, and symbols they represent. If you are using a windowing

3. The GNU Project is a project by the Free Software Foundation to produce a complete, free operating system.

system (like Microsoft Windows® or the X Window System), then the windowing system reads the keypress from the kernel, and delivers it to whatever program is currently in focus on the user's display.

Example 1-1. How the computer processes keyboard signals

Keyboard -> Kernel -> Windowing system -> Application program

The kernel also controls the flow of information between programs. The kernel is a program's gate to the world around it. Every time that data moves between processes, the kernel controls the messaging. In our keyboard example above, the kernel would have to be involved for the windowing system to communicate the keypress to the application program.

As a fence, the kernel prevents programs from accidentally overwriting each other's data and from accessing files and devices that they don't have permission to. It limits the amount of damage a poorly-written program can do to other running programs.

In our case, the kernel is Linux. Now, the kernel all by itself won't do anything. You can't even boot up a computer with just a kernel. Think of the kernel as the water pipes for a house. Without the pipes, the faucets won't work, but the pipes are pretty useless if there are no faucets. Together, the user applications (from the GNU project and other places) and the kernel (Linux) make up the entire operating system, GNU/Linux.

For the most part, this book will be using the computer's low-level assembly language. There are essentially three kinds of languages:

Machine Language

This is what the computer actually sees and deals with. Every command the computer sees is given as a number or sequence of numbers.

Chapter 1. Introduction

Assembly Language

This is the same as machine language, except the command numbers have been replaced by letter sequences which are easier to memorize. Other small things are done to make it easier as well.

High-Level Language

High-level languages are there to make programming easier. Assembly language requires you to work with the machine itself. High-level languages allow you to describe the program in a more natural language. A single command in a high-level language usually is equivalent to several commands in an assembly language.

In this book we will learn assembly language, although we will cover a bit of high-level languages. Hopefully by learning assembly language, your understanding of how programming and computers work will put you a step ahead.